

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 3

The Command Line: Shell, Pipes and Text Filters

Track A: The Real OS, from the Outside (Linux)

Elaborated by:

st. gr. FAF-241

Ermicev Sofia

Verified by:

Patricia Reîțman

Contents

1	Introduction	2
2	Part 1. Move Fast, Find Help	2
2.1	Command types and PATH	2
2.2	Observations	3
3	Part 2. What the Shell Does to the Command Line	3
3.1	Observations	4
4	Part 3. Redirection, Chaining and Jobs	5
4.1	Order of redirections	6
4.2	Input redirection and command operators	6
5	Part 4. Answering Questions with One-Liners	7
5.1	Q1. How many requests failed?	7
5.2	Q2. How many different pages were requested, and which? . . .	7
5.3	Q3. How many requests did each page get?	7
5.4	Q4. Which user or users have the most failed requests, and how many?	7
5.5	Q5. How many requests did user3 make, and how many failed? .	8
5.6	Q6. Last three failed requests, time and user only	8
5.7	Q7. Login shells in /etc/passwd	8
5.8	Q8. Why do the two ls pipelines differ by one?	8
5.9	Bonus	9
6	Part 5. First Shell Script	9
6.1	Observations	10
7	Conclusion	11

1. Introduction

The purpose of this laboratory work is to develop practical skills in using the Bash command line and to understand how the shell interprets a command before executing a program. The work covers command discovery, PATH lookup, shell expansions, quoting, redirection, command chaining, background jobs, pipelines, text filters, and basic shell scripting.

The experiments were performed in Ubuntu running under Windows Subsystem for Linux 2 (WSL2). A synthetic web access log containing 500 records was generated and used for the pipeline exercises.

```
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $(i % 5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
```

The verification command reported:

```
500 access.log
```

2. Part 1. Move Fast, Find Help

2.1. Command types and PATH

The commands `type`, `which`, and inspection of the `PATH` variable were used to determine how Bash finds commands.

```
type cd
type ls
type echo
which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog
echo "exit code: $?"
```

The observed output included:

```
cd is a shell builtin
ls is aliased to `ls --color=auto`
echo is a shell builtin
/usr/bin/ls
...
```

```
nosuchprog: command not found
exit code: 127
```

There were **49 directories** in the current PATH. The external `ls` program was located at `/usr/bin/ls`.

2.2. Observations

1. `cd` and `echo` are shell built-ins. In this Bash configuration, `ls` is an alias that ultimately invokes the external program `/usr/bin/ls`. `cd` must be a built-in because changing directory must modify the working directory of the current shell process; a separate child process cannot change its parent's working directory.
2. The PATH contained 49 directories. The `ls` executable was found in `/usr/bin`.
3. A command that is not found returned exit code **127**.
4. The `ls` option that sorts by modification time is `-t` (equivalently `--sort=time`); newer files are shown first by default.

Tab completion expanded partial path names, for example `/usr/sh` to entries under `/usr/share/` and `/et` to `/etc/`. `Ctrl+R` started reverse incremental search through shell command history.

Listing 1: Output of `type`, `which` and the missing-command exit code

```
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
nosuchprog: command not found
exit code: 127
```

3. Part 2. What the Shell Does to the Command Line

A test directory was prepared with several files:

```
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
```

The shell expansions were then tested. The prediction column records the expected behavior before interpreting the observed result.

Command	Prediction	Real output / explanation
echo *.c	Names ending in .c.	a.c b.c. Bash expanded the glob before echo ran.
echo file?.txt	Both numbered text files.	file1.txt file2.txt; ? matched one character.
echo [a-b]*	Names starting with a or b.	a.c alfa beta.txt b.c. The file containing a space was one filename even though echo printed the space normally.
echo *.xyz	No matching files.	Literal *.xyz. With Bash's default settings an unmatched glob remains unchanged.
ls alfa beta.txt	It will fail because the name contains a space.	Two separate names were passed to ls; both were missing.
ls "alfa beta.txt"	The file will be found.	'alfa beta.txt'; quoting preserved the space as part of one argument.
echo '\$HOME'	Literal, expanded, literal.	\$HOME /home/sofia \$HOME.
echo {1..5}	Numbers 1 to 5.	1 2 3 4 5; brace expansion was performed by Bash.
echo \$((7 * 6))	42.	42; arithmetic expansion.
echo "today is \$(date +%A), I am \$(whoami)"	Day and username substituted.	today is Monday, I am sofia; command substitution was evaluated inside double quotes.
touch log{1..3}.txt && ls log*	Three log files will be created and listed.	log1.txt log2.txt log3.txt.

3.1. Observations

1. The predictions and results are summarized in the table above. The key point is that many transformations are performed by Bash before the selected program starts.
2. echo did not search the filesystem. Bash performed pathname expansion (globbing) for *.c and passed the resulting filenames as arguments to echo.

3. `ls alfa beta.txt` failed because Bash split the text at the space, producing two arguments. Two valid fixes are `ls "alfa beta.txt"` and `ls alfa\ beta.txt`.
4. Single quotes preserve all enclosed characters literally. Double quotes preserve spaces but still allow expansions such as `$HOME` and `$(command)`. A backslash can escape an individual special character. `ls` may display a filename as `'alfa beta.txt'` to make the embedded space unambiguous to the reader.

Listing 2: Globbing and quoting results

```
a.c b.c
file1.txt file2.txt
a.c alfa beta.txt b.c
*.xyz
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
'alfa beta.txt'
$HOME /home/sofia $HOME
1 2 3 4 5
42
today is Monday, I am sofia
log1.txt log2.txt log3.txt
```

4. Part 3. Redirection, Chaining and Jobs

Standard output and standard error were first redirected to different files:

```
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt
cat err.txt
```

The result was:

```
184 out.txt
ls: cannot access '/nope': No such file or directory
```

Therefore, the directory listing was written to `out.txt`, while the error message was written to `err.txt`.

4.1. Order of redirections

The two forms below are not equivalent:

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

In the first form, stdout is redirected to the file and stderr is then made a duplicate of the already redirected stdout; therefore both streams go to the file. In the second form, stderr is first duplicated from the current stdout (the terminal), and only afterwards stdout is redirected to the file. Consequently, the error still appears on the terminal.

4.2. Input redirection and command operators

The command `wc -l < access.log` produced only:

```
500
```

whereas `wc -l access.log` produced:

```
500 access.log
```

The reason is that with input redirection `wc` receives data through standard input and is not given a filename argument to print.

The shell operators used in this part have the following meanings:

- `;` executes the next command regardless of the previous command's exit status.
- `&&` executes the next command only if the previous command succeeds (exit status 0).
- `||` executes the next command only if the previous command fails (non-zero exit status).

Error suppression and conditional execution produced:

```
failed with code 2
```

A background job was created with `sleep 100 &`. Bash reported a job number and PID, and `kill %1` terminated it.

Listing 3: Background job and termination

```
[1] 2384
[1]+  Running                  sleep 100 &
[1]+  Terminated              sleep 100
```

5. Part 4. Answering Questions with One-Liners

The file `access.log` contained 500 lines with fields: date, time, user, result, and page. The following results were obtained.

5.1. Q1. How many requests failed?

```
grep -c FAIL access.log
```

Result: **100** failed requests.

5.2. Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u
```

The pages were `/page0`, `/page1`, `/page2`, and `/page3`; therefore there were **4** distinct pages.

5.3. Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
```

```
125 /page3  
125 /page2  
125 /page1  
125 /page0
```

The counts sum to 500, matching the total number of log entries.

5.4. Q4. Which user or users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

The maximum was shared by **user3** and **user5**, with **15 failures each**. The remaining users each had 14 failures.

5.5. Q5. How many requests did user3 make, and how many failed?

```
grep -c ' user3 ' access.log  
grep ' user3 ' access.log | grep -c FAIL
```

user3 made **72 requests**, of which **15 failed**.

5.6. Q6. Last three failed requests, time and user only

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

```
10:10 user0  
10:15 user5  
10:20 user3
```

5.7. Q7. Login shells in /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

```
26 /usr/sbin/nologin  
2 /bin/bash  
1 /bin/sync  
1 /bin/false
```

5.8. Q8. Why do the two ls pipelines differ by one?

The commands produced:

```
ls /etc | wc -l  
183  
ls -l /etc | wc -l  
184
```

The long listing begins with an additional `total ...` line that summarizes allocated blocks. `wc -l` counts this line too, hence the difference of exactly one.

5.9. Bonus

The regular expression selecting failed requests whose minute ends in zero found **50 lines**:

```
grep -Ec '10:[0-9]0 .* FAIL' access.log
50
```

Listing 4: Main results from oneliners.sh

```
100
/page0
/page1
/page2
/page3
Count: 4
125 /page3
125 /page2
125 /page1
125 /page0
15 user5
15 user3
Total: 72
Failed: 15
10:10 user0
10:15 user5
10:20 user3
```

6. Part 5. First Shell Script

The script `report.sh` was created to summarize a supplied log file and to handle a missing input file correctly.

```
#!/bin/bash
# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
```

```
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for i in {0..6}; do
    count=$(grep " user$i " "$log" | grep -c FAIL)
    echo "user$i: $count failures"
done
```

After `chmod +x report.sh`, execution on the generated log produced:

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Running it with a missing file produced:

```
no such file: nope.log
exit code: 1
```

The final permissions were:

```
-rwxr-xr-x 1 sofia sofia 323 Oct 5 16:15 report.sh
```

6.1. Observations

1. The first line, `#!/bin/bash`, is the shebang. When the file is executed directly, it tells the operating system to use Bash as the interpreter. Before execute permission is granted with `chmod +x`, attempting `./report.sh` normally results in a permission error.
2. `./report.sh` is required because the current directory (`.`) is normally not part of `PATH`. The explicit `./` tells the shell where the executable file is located.
3. `>&2` sends the diagnostic message to standard error rather than standard output. `exit 1` returns a non-zero status so callers and scripts can detect that execution failed.

7. Conclusion

This laboratory work demonstrated that the shell is not merely a program launcher. Before execution, Bash performs command lookup, pathname expansion, quote processing, arithmetic expansion, command substitution, and redirection setup. The experiments also showed how small UNIX text-processing programs can be combined through pipelines to answer practical questions about structured text data.

The log-analysis exercises reinforced the use of `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, and `tail`. Finally, the `report.sh` script combined these command-line techniques with variables, conditionals, loops, `stderr` handling, exit codes, and executable permissions to form a small reusable Bash program.